UniView: Camera-Based Parking Occupancy Sensing for a Smart Campus

Conceptual Design Report

Team Members: Adam Derbala, Douglas Lavin, Ethan Ryder, Jeffrey Szeto
Advisors: Sasan Haghani, Shweta Agarwal, Suzanne Schnaars, Namrata Prakash
Team Number: SP26-24

***Abstract*—Project UniView is a smart parking occupancy monitoring system developed as part of the ECE Capstone program in collaboration with Verizon. The goal of this project is to design a scalable and low-cost solution capable of detecting real-time parking availability using edge-based computer-vision techniques. UniView deploys low-power edge nodes that capture periodic images of a lot section, run local computer-vision inference, and transmit compact occupancy metadata instead of video. Existing parking monitoring solutions can be accurate but often require extensive infrastructure, centralized processing, and specialized installation leading to increased deployment and operational costs. UniView instead emphasizes flexible deployment, low bandwidth usage, and privacy-by-design by avoiding storage or transmission of identifiable imagery in normal operation. The system is designed for outdoor campus deployments and emphasizes a modular architecture, allowing it to scale to multiple campus locations with minimal reconfiguration. The expected outcome is an end-to-end demonstrator covering a representative campus lot segment, with quantified accuracy and end-to-end latency under varying lighting and weather conditions. Deliverables include an edge node implementation, backend storage and REST API, a simple visualization dashboard, and a deployment plan with cost scaling, risks, and mitigation strategies for field operation. System performance will be evaluated based on occupancy accuracy, end-to-end latency, network usage per update, and robustness to connectivity loss and environmental variability. UniView is intended for use by campus drivers, as well as facilities and transportation staff who require utilization analytics to support planning and operations.**

***Index Terms*—smart campus, parking occupancy, edge AI, computer vision, IoT, LTE, Raspberry Pi**

I. Recognize the Need (Gather Information)

A. Problem Statement

Parking search time increases congestion, emissions, and driver frustration. On a university campus, the impact is amplified by peak arrivals around class start times, special events, and constrained lot layouts. Facilities and campus transportation services also benefit from data on lot utilization to inform signage, enforcement, and long-term planning. The technical need is a sensing system that can provide timely and reasonably accurate occupancy information with minimal infrastructure changes to the lot.

B. Stakeholders and Target Audience

Campus drivers (students, faculty, staff, and visitors) who need a quick and simple indication of where to park make up the primary audience. Secondary stakeholders include campus operations and transportation, who need aggregated analytics (trends by time/day/semester), maintenance visibility (node health), and the ability to scale to multiple lots.

C. Existing Approaches and Gaps

Common approaches include:

- **In-ground sensors**: generally accurate per-space detection, but requires involved installation (trenching, sealing) along with higher upfront costs and long-term maintenance.
- **Overhead cameras with central processing**: can cover many spaces, but may require stable power, network backhaul, and careful privacy handling.
- **Counting systems** (entrance/exit): simpler installation, but provide only estimated availability and drift with atypical traffic patterns.

By emphasizing low power consumption, cost efficiency, and simple installation, UniView aims to provide a practical alternative to traditional solutions that are often expensive and require specialized infrastructure.

II. Define the Scope of the Project

A. Project Goals

1) Build an end-to-end prototype that outputs occupancy states for a target lot segment (per-space or per-zone).
2) Provide a backend service that stores results, exposes an API, and supports a basic visualization dashboard.
3) Demonstrate reliable operation under representative campus conditions (day/night variation, weather, occlusions).
4) Document a deployment plan including bill of materials (BOM), risks, and a scaling strategy.

B. Design Constraints

- **Power**: support low-power operation; prototype may use wall power, but design must include solar/battery feasibility.
- **Connectivity**: tolerate limited bandwidth; prefer LTE/5G or intermittent uplink; avoid streaming video.

- **Privacy**: do not store or transmit identifiable imagery by default; retain only occupancy results and non-identifying diagnostics.
- **Environment**: operate outdoors (temperature, precipitation); enclosure and mounting must be considered.
- **Capstone timeline**: prioritize a robust MVP and measurable performance over exhaustive features.

C. Deliverables

Primary deliverables:

- Working prototype (edge node, gateway/cloud backend, dashboard).
- Test report with accuracy and latency metrics.
- System documentation (architecture, setup, operations, maintenance).

Optional deliverables (nice-to-have):

- Per-space calibration tool and semi-automated space labeling.
- Health monitoring alerts (battery level, temperature, camera obstruction).
- Navigation to specific stall location; identify preferred parking locations
- Analytics views (heatmaps, historical occupancy trends).

III. Preliminary Design

This section presents the system requirements, architecture, and design decisions for UniView. The design prioritizes computational efficiency, privacy compliance, and deployability in infrastructure-limited environments.

A. System Requirements

The following tables define the functional and non-functional requirements for the UniView system. Each requirement includes a priority level (H = High, M = Medium, L = Low) and an acceptance criterion that specifies how compliance will be verified during testing.

1) Functional Requirements: Table I lists the functional requirements that define what the system must do. These requirements cover image capture, occupancy detection, data transmission, backend services, and user interface capabilities.

TABLE I
Functional Requirements

ID	Requirement	Pri.	Acceptance Criterion
FR-01	Capture images at configurable intervals (default 30 s).	H	Image within ± 1 s of schedule; interval adjustable.
FR-02	Detect parking space boundaries from reference image.	H	$\geq$95% of ground-truth spaces detected.
FR-03	Classify each space as Occupied or Vacant.	H	Binary label per space per cycle.
FR-04	Transmit only occupancy metadata (no images).	H	Payload $\leq$1 KB; no image bytes.
FR-05	Buffer results during connectivity loss; retry on reconnect.	M	Store $\geq$24 hrs offline; upload within 5 min.
FR-06	Expose REST API for current/historical occupancy.	H	`GET /lots/{id}/status` $\leq$500 ms.
FR-07	Dashboard shows real-time lot occupancy.	H	UI updates within 5 s of backend data.
FR-08	Dashboard shows historical trends.	M	Line chart renders with date selector.
FR-09	Optional debug image upload (flag-gated).	L	Flag off $\rightarrow$ zero images.
FR-10	Report node health telemetry.	M	Health JSON visible; alert if silent $>$10 min.

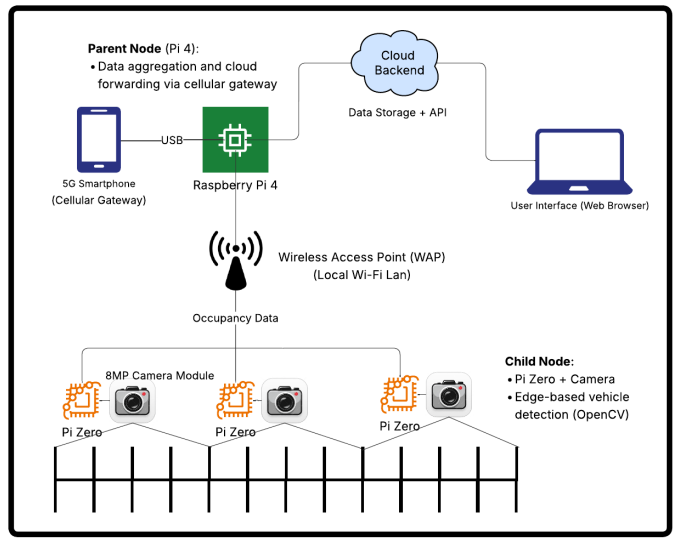

Fig. 1. System Architecture Diagram

2) Non-Functional Requirements: Table II lists the non-functional requirements that define quality attributes and constraints. These requirements specify performance targets, reliability thresholds, and operational constraints that the system must satisfy.

TABLE II
Non-Functional Requirements

ID	Requirement	Pri.	Acceptance Criterion
NFR-01	Accuracy $\geq$90% (daylight).	H	F1 $\geq$0.90 on $\geq$500 labeled frames.
NFR-02	Accuracy $\geq$80% (night, artificial light).	M	F1 $\geq$0.80 on $\geq$200 frames.
NFR-03	End-to-end latency $\leq$60 s.	H	95th pct $\leq$60 s over 1-day pilot.
NFR-04	Child node power $\leq$5 W.	M	USB power meter, 1-hr steady state.
NFR-05	Bandwidth $\leq$10 KB per update.	H	Wireshark confirms.
NFR-06	Uptime $\geq$99% over 7-day pilot.	M	$<$1.7 hrs downtime logged.
NFR-07	No PII imagery stored/transmitted.	H	Code review + privacy audit.
NFR-08	Supports $\geq$10 child nodes per parent.	M	Load test; latency increase $\leq$20%.
NFR-09	Operates -10°C to $+45$°C, IP65 enclosure.	M	Spec sheet + thermal test.
NFR-10	Docs enable onboarding in $\leq$4 hrs.	L	New member deploys node.

B. High-Level Architecture

UniView is structured as a layered system:

- **Edge node**: camera + embedded compute running capture, preprocessing, and occupancy inference.
- **Gateway/parent**: aggregates multiple nodes (optional), manages configuration updates, and buffers data.
- **Backend**: receives occupancy events, stores time series data, and exposes REST endpoints.
- **Frontend**: dashboard showing current availability and historical trends.

C. Subsystems

Imaging subsystem: fixed-position camera with a field of view covering a set of spaces. Exposure settings and IR/night

performance are considered in testing.

Edge AI subsystem: two candidate approaches are considered:

- **Classical CV**: background modeling + ROI occupancy classification per space.
- **ML-based detection**: lightweight object detection/segmentation model fine-tuned for vehicles and space boundaries.

The MVP will prioritize a classical computer vision pipeline that is robust and maintainable on constrained hardware, as detailed in Section III-D.

Communications subsystem: occupancy results are encoded as compact messages (e.g., JSON) containing node ID, timestamp, and an occupancy bitmap or per-space states. Images are not transmitted in normal operation; optional debug uploads are gated behind an explicit flag.

Backend and data subsystem: stores occupancy events and derived metrics (occupancy percentage, turnover). A simple schema supports querying by lot and time window.

D. Algorithms and Data

Occupancy inference is performed on the edge node using classical computer vision techniques selected for computational efficiency and deterministic behavior. This design avoids machine learning models during initial deployment in order to minimize memory usage, power consumption, and operational complexity on low-cost embedded hardware.

The computer vision pipeline is divided into an initialization phase and a real-time inference phase.

1) Initialization Phase: During initialization, each edge node captures a reference image of the parking area within its field of view. This image is used to define parking space geometry and establish a baseline for change detection.

1) **Grayscale Conversion:** The reference image is converted to grayscale by averaging the RGB channels, reducing dimensionality and preprocessing cost.
2) **Edge Detection:** Canny edge detection is applied to the grayscale image to identify structural boundaries such as parking lines and curbs [1]. Prior to gradient computation, Gaussian smoothing is applied to suppress sensor noise.
3) **Line and Region Extraction:** Detected edges are grouped using regression-based fitting to form line segments corresponding to parking boundaries. These lines are combined to define quadrilateral regions representing individual parking spaces.

The resulting set of parking space polygons is stored locally on the node and reused during real-time inference.

2) Real-Time Occupancy Inference: During normal operation, edge nodes periodically capture new images and compare them against the reference state.

1) **Image Differencing:** The absolute pixel-wise difference between the current image and the reference image is computed to highlight scene changes.
2) **Edge Detection on Difference Image:** Canny edge detection is applied to the difference image to isolate regions of high contrast corresponding to vehicle presence.
3) **Vehicle Region Estimation:** Edge clusters are approximated using polygon fitting to estimate vehicle footprints. Perspective projection based on camera mounting height and viewing angle is used to map detected regions onto the ground plane.
4) **Occupancy Classification:** A parking space is classified as occupied if the projected vehicle polygon overlaps the predefined space polygon beyond a fixed threshold.

The output of each inference cycle is an occupancy bitmap indexed by parking space, along with optional confidence metrics and basic node health telemetry.

3) Data Products: Key data outputs include:

- **Real-time data:** per-space occupancy states and update latency.
- **Offline data:** aggregated utilization metrics, trend analytics, and accuracy measurements derived from labeled evaluation sets.

E. Cloud & UI

The cloud backend and web-based user interface enable parking occupancy data generated at the edge to be stored, retrieved, and visualized in real time.

1) Data Flow Overview: Occupancy detection is performed entirely on edge hardware. Individual child nodes detect stall occupancy and transmit compact results to a parent node. The parent node aggregates data across multiple child nodes within a parking lot and forwards the aggregated results to the cloud backend over a network connection (campus WiFi or cellular) using standard HTTPS protocols.

2) Technology Stack: The backend is implemented using Python-based technologies to maintain consistency with the edge node software:

- **API Framework:** FastAPI, providing automatic OpenAPI documentation, type validation, and asynchronous request handling.
- **Database:** PostgreSQL for production; SQLite for local development.
- **ORM:** SQLAlchemy for database abstraction and schema management.
- **Visualization:** Google Maps JavaScript API for lot geometry rendering; Chart.js for historical trend charts.
- **Hosting:** Railway, Render, or Fly.io (free tier sufficient for prototype).

3) Data Model: The database stores only compact occupancy metadata. Image data is not transmitted or retained during normal operation, supporting privacy requirements and reducing bandwidth and storage overhead. Stored information includes:

- **Lots:** Parking lot identifiers, names, geographic coordinates, and total stall counts.
- **Stalls:** Individual stall identifiers, lot association, and position coordinates (latitude/longitude or pixel offsets for map overlay).

- **Occupancy Events:** Stall occupancy state (occupied or available), node identifier, and timestamp of each update.

Occupancy events are indexed by timestamp and lot identifier to support efficient queries for both real-time status and historical analysis.

4) API Endpoints: Table III summarizes the REST API endpoints exposed by the backend.

TABLE III
REST API ENDPOINTS

Method	Endpoint	Description
POST	`/api/v1/occupancy`	Ingest occupancy update from parent node
GET	`/api/v1/lots`	List all monitored parking lots
GET	`/api/v1/lots/{id}/status`	Current stall-level occupancy for a lot
GET	`/api/v1/lots/{id}/history`	Historical occupancy data (time range)
GET	`/api/v1/nodes/{id}/health`	Node health and last update time

All endpoints return JSON payloads. When a user selects a parking lot on the website, the backend queries the database for the most recent occupancy state of each stall within that lot, along with the corresponding stall position information. The retrieved data is returned in a structured format that allows the frontend to match occupancy states to specific stall positions on the map.

5) Web-Based User Interface: The user interface is implemented as an interactive web-based application. Users can open the website, select a parking lot, and view real-time stall availability. The interface provides:

- **Lot Selection:** List or map view of all monitored parking lots with summary occupancy percentages.
- **Stall-Level Visualization:** Parking lot layout rendered using the Google Maps JavaScript API, with occupancy data overlaid to highlight individual stalls as available (green) or occupied (red).
- **Historical Trends:** Line charts showing occupancy percentage over selectable time ranges (hourly, daily, weekly), rendered using Chart.js.
- **Node Health Status:** Indicators showing last update time and connectivity state for each edge node (primarily for operators and debugging).

Third-party mapping services enable rendering of parking lot geometry and user interaction without requiring custom map development. Data access is designed to be simple and efficient, focusing on delivering the latest occupancy information needed for visualization. Historical data is stored primarily for analysis and evaluation purposes and is not required for basic real-time operation of the user interface.

6) Security Considerations:

- **Transport:** All communication between parent nodes and the backend uses HTTPS (TLS 1.2+).
- **Authentication:** Node-to-backend ingestion uses API key authentication; dashboard access is unauthenticated for the prototype.
- **Input Validation:** FastAPI's Pydantic models enforce schema validation on all incoming data.
- **Privacy:** No personally identifiable information is collected. Only stall identifiers, binary occupancy states, and timestamps are stored.

F. Testing Strategy

Testing will be staged:

- **Bench tests**: validate camera capture, inference speed, and message formatting.
- **Controlled lot tests**: collect labeled frames across lighting/weather scenarios; compute accuracy.
- **Field pilot**: multi-day deployment; validate uptime, drift, and maintenance needs.

Performance metrics include occupancy accuracy, end-to-end latency, power draw, and network usage per node.

G. Potential Solutions Considered

Table IV summarizes candidate solution paths and trade-offs.

TABLE IV
CANDIDATE SOLUTION OPTIONS

Option	Approach	Notes
A	Classical CV per space	Low compute cost; sensitive to shadows/lighting; simpler to explain and debug.
B	Lightweight ML detector	Better generalization; requires labeled data and model management.
C	Counting at entrances	Lowest install complexity; does not provide per-space availability.

Machine learning–based approaches, such as convolutional neural networks for object detection, were considered due to their ability to generalize across lighting and environmental conditions [2]. However, these methods require labeled datasets, model retraining, and substantially greater compute and memory resources, which conflict with low-power edge deployment goals.

Alternative sensing approaches, including entrance-based counting and non-visual sensors, reduce computational complexity but fail to provide per-space availability or require dense hardware installation [3]. Based on these trade-offs, classical computer vision methods were selected for the initial system implementation.

IV. PLAN THE PROJECT

A. Work Breakdown and Team Roles

Work is divided into parallel tracks:

- Edge node software (capture, inference, telemetry)
- Backend/API + database
- Dashboard UI + analytics views
- Hardware integration (mounting, enclosure, power, networking)
- Testing + data labeling + evaluation

B. Timeline (Preliminary Gantt Chart)

Figure 2 presents the preliminary project timeline for UniView, outlining the major development phases and milestones across the capstone period. The schedule reflects a high-level plan and may be refined as system requirements and testing results evolve.

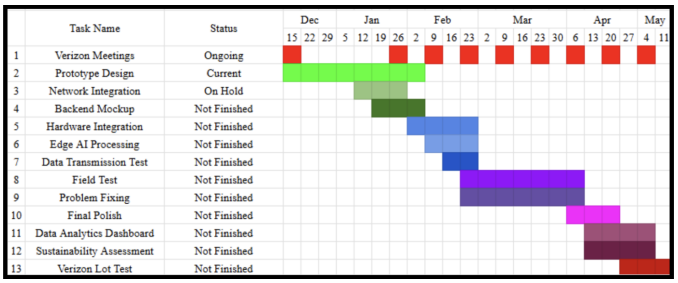

Fig. 2. Project development timeline (Gantt chart)

C. Cost Estimation

Table V provides a prototype Bill of Materials and identifies all hardware required for the development of Uniview. Additional funding will be pursued if available and efforts will be made to use owned or recycled materials in order to reduce cost.

TABLE V
PROTOTYPE BILL OF MATERIALS

Category	Item	Qty	Unit ($)	Subtotal ($)	Status	Source
Child Nodes (×3)						
	Raspberry Pi Zero 2 W	3	15.00	45.00	To Purchase	Adafruit
	Pi Camera Module 3 (Wide)	3	35.00	105.00	To Purchase	Adafruit
	Zero Camera Cable (15 cm)	3	6.00	18.00	To Purchase	Adafruit
	MicroSD Card (16 GB)	3	8.00	24.00	To Purchase	Amazon
	Micro-USB Power Supply (5V 2.5A)	3	8.00	24.00	To Purchase	CanaKit
Parent Node (×1)						
	Raspberry Pi 4 Model B (4 GB)	1	55.00	0.00	**Owned**	—
	MicroSD Card (32 GB)	1	12.00	0.00	**Owned**	—
	USB-C Power Supply (5.1V 3A)	1	10.00	0.00	**Owned**	—
	Pi 4 Case with Heatsinks	1	15.00	0.00	**Owned**	—
Networking						
	WiFi Router (local LAN for children)	1	0.00	0.00	**Owned**	—
	Rutgers WiFi / Ethernet Uplink	—	0.00	0.00	Available	Campus
Enclosures & Mounting						
	IP65 Outdoor Enclosure (child)	3	18.00	54.00	To Purchase	Amazon
	IP65 Outdoor Enclosure (parent)	1	25.00	25.00	To Purchase	Amazon
	Pole Mount Brackets	4	10.00	40.00	To Purchase	Amazon
	Misc. Cables, Zip Ties, Grommets	1	15.00	15.00	To Purchase	Home Depot
Power						
	Outdoor Extension Cord (50 ft)	1	25.00	25.00	To Purchase	Home Depot
	Outdoor Power Strip	1	20.00	20.00	To Purchase	Amazon
Development (One-Time)						
	HDMI Monitor	1	—	0.00	**Owned**	—
	USB Keyboard + Mouse	1	—	0.00	**Owned**	—
	HDMI Cables	2	—	0.00	**Owned**	—
Items Already Owned				**$92.00**	(not charged)	
New Purchases Required				**$395.00**		
Project Budget				**$395.00**		

D. Risks and Roadblocks

Primary risks:

- **Lighting variability**: mitigate with data collection and normalization; consider IR/night mode.
- **Occlusion and snow/rain**: define acceptable failure modes; include health checks for obstruction.
- **Compute limits**: profile inference; adjust frame rate and model size.
- **Mounting and field of view**: plan adjustable mounts and calibration procedure.
- **Connectivity outages**: buffer results locally; upload on reconnect.

E. Conclusion

Project UniView proposes a scalable, low-cost smart parking occupancy monitoring system that leverages edge-based computer-vision techniques to provide real-time parking availability while minimizing bandwidth usage and preserving user privacy. By shifting inference to low-power edge nodes and transmitting compact occupancy metadata, the system aims to reduce infrastructure complexity and operational costs compared to centralized parking monitoring approaches. The modular, campus-oriented design is intended to support flexible deployment across multiple locations.

The next steps for this project include implementing a prototype edge node and end-to-end data pipeline, followed by integration of backend services and a basic visualization dashboard. Subsequent phases will involve deploying the system in a representative campus parking area and evaluating performance under varying environmental and connectivity conditions. These steps will establish a foundation for validating system accuracy, latency, robustness, and scalability.

ACKNOWLEDGMENT

The authors would like to acknowledge the support and guidance provided by Verizon throughout the UniView capstone project. The authors also thank their faculty advisor, Dr. Sasan Haghani, for his continued mentorship and oversight during the conceptual design phase, as well as the engineering mentors for their valuable technical feedback and insights throughout the project.

REFERENCES

[1] J. Canny, "A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[3] L. Baroffio, L. Bondi, M. Cesana, A. Redondi, and M. Tagliasacchi, "A visual sensor network for parking lot occupancy detection in smart cities," *IEEE Internet of Things Journal*, vol. 2, no. 5, pp. 1–10, 2015.